

P1749R0

Access control for reflection

Draft Proposal, 2019-06-14

This version:

https://yehezkelshb.github.io/cpp_proposals/P1749-access-control-for-reflection.html

Author:

[Yehezkel Bernat](#)

Audience:

SG7 (REFLECTION)

Project:

ISO/IEC JTC1/SC22/WG21 14882: Programming Language — C++

Abstract

P1749 claims that reflection must be constrained in regular code and obey access control and scope rules

Table of Contents

1 Problem statement

- 1.1 ODR - One *Declaration* Rule
- 1.2 Backward compatibility and API breaking
- 1.3 Portability and standardization issues
- 1.4 Effect on readability
- 1.5 Encouraging bad code techniques
- 1.6 Inconsistency with other parts of the standard

2 Proposed solution

3 Acknowledgements

4 References

§ 1. Problem statement

In "The Design & Evolution of C++", there is a section about "Keyword Arguments" (section 6.5.1, page 153), a language extension proposed in the process of the first standardization of C++. It was about "a mechanism for specifying function arguments by name" in the call site, e.g. `new window(Color:=green, ysize:=150);`. Bjarne Stroustrup mentioned there were a few concerns and issues, which were the reasons to reject this extension (pg. 155 and on). Reading it, the author found that some of the arguments are still relevant today and apply to reflection.

Here are some of those arguments and some comments, perspectives and thoughts about them and how they are relevant for reflection.

§ 1.1. ODR - One *Declaration* Rule

The first serious problem discovered with the proposal was that keyword arguments would introduce a new form of binding between a calling interface and an implementation:

[1] An argument must have the same name in a function declaration as in the function definition.

[...]

Worse, this turned out to be a compatibility problem of significant magnitude. Some organizations recommend a style with "long, informative" argument names in header files, and "short, convenient" names in the definitions. For example:

```
void reverse(int* elements, int length_of_elements_array);  
// ...  
void reverse(int* v, int n)  
{  
    // ...  
}
```

Naturally, some people find that style abhorrent, whereas others (including me) find it quite reasonable. Apparently, significant amounts of such code exist.

[p0670r2](#) says:

This concern is mitigated in two ways:

1. Modern coding conventions have the declarations for a particular function showing up in exactly one header file.
2. Modern coding conventions discourage the use of different argument names between function declarations (in a header) and function definitions (in a '.cpp' file). Dedicated compiler warnings exist to protect against this case.

So we don't think modern code is using such a coding guideline anymore. The question is why we would want to add a new way of "ODR-violation" sort of issues into the language, and help the potentially breakage of older code that was written to these guidelines.

Alternatively, the language shouldn't require declarations to have the same name for the same argument. That seemed viable to me. However, people didn't seem to like that variant either.

There could be a noticeable impact on link times if the rule that arguments names must match across compilation units is checked. If it isn't checked, the facility would not be type safe and could become a source of subtle errors.

So he considered such facility type unsafe. An interesting point.

§ 1.2. Backward compatibility and API breaking

[2] Once a keyword argument is used, the name of that argument cannot be changed in the function definition without breaking user code.

[...]

Further, an implication of keyword arguments would be that no name in a commonly distributed head file could be changes without risking breaking code.

We are making the parameter names part of the public API of any library. Library vendors will not be able to fix a typo in the parameter name anymore, as this will be a breaking change for the users.

You may argue that any mistake in the API is hard to fix. This is true, but usually we can have them both. We can keep the older function name with the typo and introduce a new function with the correct name (probably `[[deprecated]]`-ing the old one). We can add a type alias for a mistake in the type name. We can add a new overloading for the function with stronger types for the arguments. None of

these solutions is relevant for argument names. We'll have to introduce a new function name (FuncEx, anyone?) just to fix the parameter name!

§ 1.3. Portability and standardization issues

Different suppliers of header files for common services (for example, Posix or X) would also have to agree on argument names. This could easily become a bureaucratic nightmare.

Do we want to specify the parameter names of all the standard library functions and force them on the implementations? [SD-8](#) tries hard to free the standard library from various possible backward compatibility issues, and allowing reflection on parameter names seems like adding a new one.

§ 1.4. Effect on readability

Both the potential linking cost and the very real binding problem could be easily avoided by omitting argument names in header files. A cautious user might therefore avoid specifying argument names in header files. Thus, to quote Bill Gibbons, "The net impact on readability of C++ might actually be negative."

Considering the previous 2 points, I think this one talks for itself. Of course, for template functions, even this escape hatch isn't available...

§ 1.5. Encouraging bad code techniques

My main worry about keyword arguments was actually that keyword arguments might slow the gradual transition from traditional programming techniques to data abstraction and object-oriented programming in C++. In code that I find best written and easiest to maintain, long argument lists are very rare. In fact, it is a common observation that a transition to a more object-oriented style leads to a significant decrease in the length of argument lists; [...]

[...]

A further reduction in the number of arguments could be obtained by using a [Point](#) type rather than expressing interfaces directly in terms of coordinates.

p0670r2 brings as an example the following code:

```
double Gauss(double x, double mean, double width, double height);
```

Do we want to encourage this style? Don't we want to teach that usage of strong types? This is true now more than ever as Metaclasses will allow easy creation of strong typedefs, without resorting to more mouthful library-based options.

§ 1.6. Inconsistency with other parts of the standard

p0542 explicitly permits redeclaration of a function with contracts to have different naming for the arguments:

```
int f(int x)
  [[expects: x>0]]
  [[ensures r: r>0]];

int f(int y)
  [[expects: y>0]]    // Should be OK
  [[ensures z: z>0]]; // Should be OK
```

This seems inconsistent, where in one part of the standard we explicitly allow different naming of function arguments and in another part we forbid it.

§ 2. Proposed solution

The solution proposed here is to allow reflection in a regular code to access only entities that are already accessible from the same code. Specifically it means:

- Parameter names are accessible from inside the function only (allowing the use-case of generating diagnostic messages)
- The regular access control is applied when accessing class members

The last point comes from the observation that API issues mentioned above are relevant for the reflection on non-public class members too.

What will be allowed is to reflect on everything (including private members and parameter names) from a metfunction or metaclass. This allows all the use-cases of generating various language bindings etc. while preventing all the mentioned issues from regular user code.

Metaclass is a different creature anyway. It's usually used by the type author, and even if we allow the `.as` operator to apply a metaclass on already existing type, this is a special operation that is known to be more fragile.

§ 3. Acknowledgements

Thanks for David Sankel for listening to my arguments and encouraging me to write this paper.

§ 4. References

- [p0670r2](#) - Static reflection of functions
- [p0542](#) - Support for contract based programming in C++
- [\SD-8](#) - Standard Library Compatibility